Projet SIO-Shoes : site « e-commerce » avec Symfony

Partie 9 : gestion du stock des produits

1. Ajouter le stock des produits.

Modifiez l'entité « Product » en exécutant la commande
`symfony console make:entity Product`
Ajoutez un champ « stock » de type « integer » (non NULL) :

```
PS C:\Symfony\e-commerce> symfony console make:entity Product
Your entity already exists! So let's add some new fields!

New property name (press <return> to stop adding fields):
> stock

Field type (enter ? to see all types) [string]:
> integer

Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/Product.php

Add another property? Enter the property name (or press <return> to stop adding fields):
>


 Success!

Next: When you're ready, create a migration with symfony.exe console make:migration
```

2. Préparez et exécutez la migration pour mettre à jour la BDD :

```
symfony console make:migration
```

```
symfony console doctrine:migrations:migrate
```

Vous pouvez voir le résultat dans phpMyAdmin : la table « product » possède maintenant un champ « stock » avec zéro comme valeur par défaut.



3. **Modifiez ensuite la fonction « BuildForm() » du formulaire « src\Form\ProductForm.php » pour ajouter le champ « stock » :**

```
$builder
->add('name')
          ->add('description')
      ->add('price')
      ->add('stock')
```

4. **Modifiez la vue « templates\product_form.html.twig » pour afficher le champ « stock ».** Attention, il ne faut pas permettre la mise à jour du stock lors de la modification d'un produit. Par contre, il doit être possible de saisir un stock pour un nouveau produit. On testera pour cela le libellé du bouton de validation « button_label » (défini dans « templates\product\edit.html.twig ») :

```
{{ form_start(form) }}

    <label for="">Nom du produit : </label>
{{ form_widget(form.name,{'attr':{'class':'form form-control','placeholder":"nom"}}) }}
    {{ form_errors(form.name) }}
    <br>
    <label for="">Description : </label>
    {{ form_widget(form.description,{'attr':{'class':'form form-control'}}) }}
    <br>
    <label for="">Prix : </label>
    {{ form_widget(form.price,{'attr':{'class':'form form-control'}}) }}
    <br>
    {% if button_label is defined and button_label == 'Modifier' %}
         <label for="">Stock : </label>
{{ form_widget(form.stock,{'attr':{'class':'form form-control','readonly': true}}) }}
    {% else %}
        <label for="">Stock : </label>
{{ form_widget(form.stock,{'attr':{'class':'form form-control','readonly': false}}) }}
{% endif %}
    <br>
    <label for="">Sous-catégories : </label>
    {{ form_widget(form.subCategories,{'attr':{'class':'form form-control'}}) }}
    <br>
    <label for="">Image : </label>
    {{ form_widget(form.image,{'attr':{'class':'form form-control'}}) }}
    <br>

    <button class="btn btn-primary">{{ button_label|default('Enregistrer') }}</button>
{{ form_end(form) }}
```

5. **Afficher le stock dans la vue « templates\product\index.html.twig », en indiquant quand le stock est épuisé.**

```
<table class="table">
    <thead>
            <tr>
                    <th>Id</th>
                    <th>Nom</th>
                    <th>Description</th>
                    <th>Prix</th>
                    <th>Stock</th>
                    <th>Actions</th>
            </tr>
    </thead>
    <tbody>
    {% for product in products %}
    <tr>
            <td>{{ product.id }}</td>
            <td>{{ product.name }}</td>
            <td>{{ product.description|slice(0,100) }} ...</td>
            <td>{{ product.price }}</td>

            {% if product.stock > 0 %}
                    <td>{{ product.stock }}</td>
            {% else %}
            <td>
                    <span class="text-danger">Stock épuisé</span>
            </td>
            {% endif %}
             <td>
               <a class="btn btn-outline-primary" href="{{ path('app_product_show', {'id':
               product.id}) }}">Afficher</a>
               <a class="btn btn-outline-secondary" href="{{ path('app_product_edit', {'id':
               product.id}) }}">Modifier</a>
            </td>
    </tr>
    {% else %}
            <tr>
                    <td colspan="6">Aucun produit à afficher</td>
            </tr>
    {% endfor %}
    </tbody>
</table>
```

6. Ajouter la gestion de l'historique du réapprovisionnement.

Créez une entité « Product » en exécutant la commande
`symfony console make:entity AddProductHistory`

Ajoutez un champ « product » de type « ManyToOne » relié à la classe « Product » (non NULL). **Répondez « Yes » à la question « Do you want to automatically delete orphaned App\Entity\AddProductHistory objects… ? »** :

```
PS C:\Symfony\e-commerce> symfony console make:entity AddProductHistory
 created: src/Entity/AddProductHistory.php
 created: src/Repository/AddProductHistoryRepository.php

 Entity generated! Now let's add some fields!
 You can always add more fields later manually or by re-running this command.

 New property name (press <return> to stop adding fields):
 > product

 Field type (enter ? to see all types) [string]:
 > ManyToOne

 What class should this entity be related to?:
 > Product

 Is the AddProductHistory.product property allowed to be null (nullable)? (yes/no) [yes]:
 > no

 Do you want to add a new property to Product so that you can access/update AddProductHistory objects from it - e.g.
 $product->getAddProductHistories()? (yes/no) [yes]:
 >

 A new property will also be added to the Product class so that you can access the related AddProductHistory objects
 from it.

 New field name inside Product [addProductHistories]:
 >

 Do you want to activate orphanRemoval on your relationship?
 A AddProductHistory is "orphaned" when it is removed from its related Product.
 e.g. $product->removeAddProductHistory($addProductHistory)

 NOTE: If a AddProductHistory may *change* from one Product to another, answer "no".

 Do you want to automatically delete orphaned App\Entity\AddProductHistory objects (orphanRemoval)? (yes/no) [no]:
 > yes

 updated: src/Entity/AddProductHistory.php
 updated: src/Entity/Product.php
```

Ajoutez ensuite un champ « **qte** » de type Integer (non NULL), puis un champ « **createdAt** » de type « datetime_immutable » (non NULL) :

```
 Add another property? Enter the property name (or press <return> to stop adding fields):
 > qte

 Field type (enter ? to see all types) [string]:
 > integer

 Can this field be null in the database (nullable) (yes/no) [no]:
 >

 updated: src/Entity/AddProductHistory.php

 Add another property? Enter the property name (or press <return> to stop adding fields):
 > createdAt

 Field type (enter ? to see all types) [datetime_immutable]:
 >

 Can this field be null in the database (nullable) (yes/no) [no]:
 >

 updated: src/Entity/AddProductHistory.php

 Add another property? Enter the property name (or press <return> to stop adding fields):
 >


  Success!
```

Remarque : le fait d'utiliser le mot « At » dans un champ permet à symfony de détecter le type d'objet et de proposer automatiquement le type « datetime_immutable ».

7. **Préparez et exécutez la migration pour mettre à jour la BDD :**

   ```
   symfony console make:migration
   ```

   ```
   symfony console doctrine:migrations:migrate
   ```

   Vous pouvez voir le résultat dans phpMyAdmin : une table « add_product_history » a été créée. Le champ « Product_id » est une clé étrangère qui référence le champ « id » de la table « Product ».



8. **Ajouter le stock initial lors de la création du produit.**

   Commencez par importer dans le contrôleur « src\Controller\ProductController.php » la nouvelle entité « AddProductHistory » : `use App\Entity\AddProductHistory;`

   Modifiez ensuite la fonction « new() » en ajoutant le code suivant juste après l'enregistrement du produit :

```
…
$entityManager->persist($product);
$entityManager->flush();

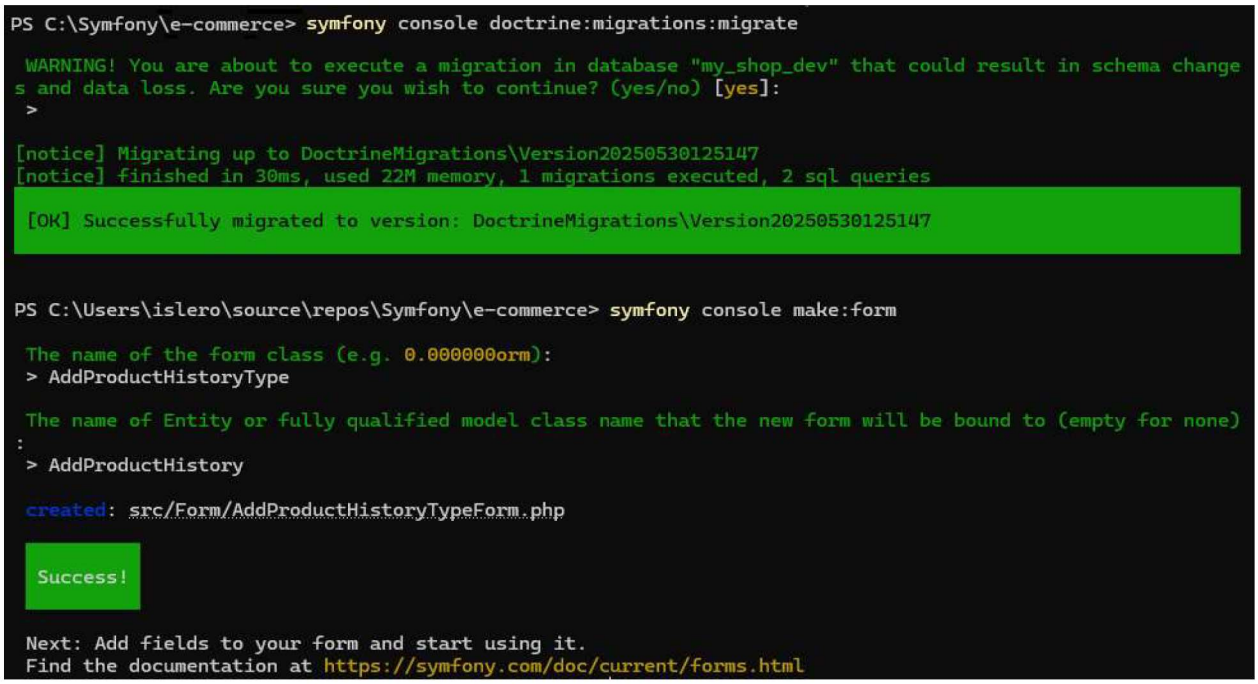
//Début du nouveau code
//Ajout du stock initial lors de la création du produit
$stockHistory = new AddProductHistory();
$stockHistory->setQte($product->getStock());
$stockHistory->setProduct($product);
$stockHistory->setCreatedAt(new \DateTimeImmutable());

$entityManager->persist($stockHistory);
$entityManager->flush();
//Fin du nouveau code
```

9. **Créer un formulaire pour mettre à jour le stock d'un produit :**

   ```
   symfony console make:form
   ```

   Nommez le formulaire « AddProductHistoryType » et reliez-le à l'entité « AddProductHistory ».



10. **Modifier la fonction « buildForm » du formulaire « src\Form\AddProductHistoryTypeForm.php » pour ne garder que le champ « qte » (on va récupérer directement le produit et la date au moment de l'enregistrement) :**

```php
public function buildForm(FormBuilderInterface $builder, array $options): void
{
        $builder
            ->add('qte')
        ;
}
```

11. **Créer une nouvelle vue « templates\product\addStock.html.twig » avec le code suivant :**

```twig
{% extends 'base.html.twig' %}
{% block title %}Ajout de produits au stock{% endblock %}
{% block body %}
<div class = "container">
    <br>
    <h1>Ajouter au stock de {{product.name}}</h1>
    <br>
   {{form_start(form)}}

        <label for="">Quantité : </label>
        {{ form_widget(form.qte,{'attr':{'class':'form form-control'}}) }}
    <br>
    <button class="btn btn-success">{{ button_label|default('Enregistrer') }}</button>
    {{form_end(form)}}
</div>
{% endblock %}
```

12. **Créer une nouvelle route dans le contrôleur « src\Controller\ProductController.php » pour mettre à jour le stock.**

Importez dans le contrôleur la bibliothèque « AddProductHistoryTypeForm » :

```
use App\Form\AddProductHistoryTypeForm
```

Ajoutez ensuite au contrôleur la route suivante :

```
#[Route('/add/product/{id}/stock', name: 'app_product_stock_add', methods: ['GET', 'POST'])]

public function addStock($id,EntityManagerInterface $entityManager,Request
$request,ProductRepository $productRepository):Response
{
    $addStock =new AddProductHistory;
    $form=$this->createForm(AddProductHistoryTypeForm::class,$addStock);
    $form->handleRequest($request);

    $product = $productRepository->find($id);
    if($form->isSubmitted()&&$form->isValid())
    {
        if($addStock->getQte()>0)
        {
            $newQte = $product->getStock() + $addStock->getQte();
            $product->setStock($newQte);

            $addStock->setCreatedAt(new \DateTimeImmutable());
            $addStock->setProduct($product);

            $entityManager->persist($addStock);
            $entityManager->flush();

            $this->addFlash('success',"Le stock du produit a été modifié !");

            return $this->redirectToRoute('app_product_index');
        }
        else
        {
            $this->addFlash('danger','Impossible de mettre à jour le stock !
            la quantité doit être > 0');
            $this->redirectToRoute('app_product_stock_add',['id'=>$product->getId()]);
        }
    }
    return $this->render('product/addStock.html.twig',[
    'form'=> $form->createView(),
    'product'=> $product
    ]);
}
```

13. **Modifier la vue « templates\product\index.html.twig » pour ajouter le bouton « Ajout stock ».**

Ajouter le code suivant juste après le lien vers la modification :

```
<a class="btn btn-outline-info" href="{{ path('app_product_stock_add', {'id': product.id})
}}">Ajout stock</a>
```

14. **Afficher l'historique des ajouts au stock des produits.**

Importez la bibliothèque « AddProductHistoryRepository » dans le contrôleur « src\Controller\ProductController.php » :

```
use App\Repository\AddProductHistoryRepository;
```

Ajoutez ensuite une nouvelle route dans le contrôleur :

```
#[Route('/add/product/{id}/stock/history', name: 'app_product_stock_add_history', methods:
['GET'])]
public function productAddHistory($id,ProductRepository $productRepository,
AddProductHistoryRepository $addProductHistoryRepository ): Response
{
    $product = $productRepository->find($id);
    $productAddedHistory = $addProductHistoryRepository->findBy(
        ['product'=>$product],['id'=>'DESC']
    );

    return $this->render('product/addedStockHistoryShow.html.twig', [
        'productsAdded' => $productAddedHistory,
        'product' => $product,
    ]);
}
```

15. **Créer la vue « templates\product\addedStockHistoryShow.html.twig » :**

```
{% extends 'base.html.twig' %}

{% block body %}
    <div class = "container">
        <table class="table">
            <thead>
                <tr>
                    <th>Id</th>
                    <th>Quantité</th>
                    <th>Date ajout</th>
                </tr>
            </thead>
            <tbody>
            {% for productAdded in productsAdded %}
```

```
            <tr>
                <td>{{ productAdded.id }}</td>
                <td>{{ productAdded.qte }}</td>
                <td>{{ productAdded.createdAt|date('d/m/Y H:i:s') }} </td>
            </tr>
        {% else %}
            <tr>
                <td colspan="3">Aucun historique à afficher</td>
            </tr>
        {% endfor %}
        </tbody>
    </table>
    <br>
     <a class="btn btn-dark" href="{{ path('app_product_show', {'id': product.id})
     }}">Retour au produit</a>
</div>
{% endblock %}
```

16. **Ajouter un lien dans la vue « templates\product\show.html.twig » pour afficher la liste des ajouts au stock :**

```
…
<a href="{{ path('app_product_stock_add_history', {'id': product.id}) }}">Afficher
l'historique des ajouts au stock</a>
<br>
<br>
<a class="btn btn-outline-primary" href="{{ path('app_product_edit', {'id': product.id})
}}">Modifier</a>
<br><br>
{{ include('product/_delete_form.html.twig') }}
<br>
<a class="btn btn-dark" href="{{ path('app_product_index') }}">Retour</a>
```